

Практическая работа №4

Теоретическое введение

Асинхронная коммуникация между микросервисами является ключевым аспектом современных распределенных приложений. В отличие от синхронной коммуникации, где каждый запрос блокирует вызывающий процесс до получения ответа, асинхронная коммуникация позволяет продолжать выполнение других операций во время ожидания ответа от удаленного сервиса. Это повышает отзывчивость системы и позволяет эффективно использовать ресурсы.

Основные преимущества асинхронной коммуникации между микросервисами:

Отзывчивость: Асинхронная коммуникация позволяет обрабатывать множество запросов параллельно без блокировки вызывающего процесса, что повышает отзывчивость системы.

Масштабируемость: поскольку асинхронные операции не блокируют поток исполнения, микросервисы могут эффективно использовать ресурсы и масштабироваться горизонтально для обработки большого числа запросов.

Устойчивость к сбоям: Асинхронная коммуникация позволяет легче обрабатывать временные сбои или задержки в сети, поскольку она не блокирует вызывающий процесс и позволяет легко обрабатывать ошибки и повторные попытки.

Гибкость: Асинхронная коммуникация позволяет создавать гибкие и отзывчивые системы, способные адаптироваться к изменяющимся условиям и требованиям.

Для реализации асинхронной коммуникации между микросервисами часто используются асинхронные протоколы и технологии, такие как

HTTP/2, WebSockets, AMQP (Advanced Message Queuing Protocol), gRPC и другие. Эти протоколы позволяют эффективно обмениваться данными между микросервисами, поддерживая асинхронные операции и обработку потока сообщений.

Полезные ссылки

1. Официальная документация Docker: <https://docs.docker.com/>
2. Docker Hub: <https://hub.docker.com/>
3. Docker Community Forums: <https://forums.docker.com/>
4. Микросервисная архитектура: введение и основные концепции:
<https://microservices.io/>
5. Использование асинхронного обмена сообщениями для улучшения
доступности: <https://habr.com/ru/companies/piter/articles/458344/>

Задание

Цель работы: реализовать асинхронную коммуникацию между микросервисами.

Данная практическая работа направлена на изучение асинхронной коммуникации между микросервисами. Асинхронная коммуникация позволяет обрабатывать большее количество запросов и повысить отзывчивость системы.

В ходе выполнения практики могут быть использованы любые языковые средства и фреймворки, для создания микросервисов.

Для примера будет использоваться асинхронный вариант веб-приложения из предыдущей практики.

Пример файла `task_service.py`:

```
import aiohttp
from aiohttp import web

async def create_task(request):
    user_id = request.match_info.get('user_id')
    task_data = await request.json()

    async with aiohttp.ClientSession() as session:
        url = f'http://user_service:5002/user/{user_id}'
        async with session.get(url) as resp:
            user_info = await resp.json()
            print("User info:", user_info)
            # Здесь можно использовать информацию о
            # пользователе для создания задачи
            return web.json_response({'message': 'Task
created successfully'})

app = web.Application()
app.router.add_post('/task/{user_id}', create_task)
web.run_app(app, port=5001, host='0.0.0.0')
```

Пример файла `user_service.py`:

```
from aiohttp import web
```

```

    async def handle_user(request):
        user_id = request.match_info.get('user_id')
        user_info = {'user_id': user_id, 'username':
'test_user'}
        return web.json_response(user_info)

app = web.Application()
app.router.add_get('/user/{user_id}',
handle_user)
web.run_app(app, port=5002, host='0.0.0.0')

```

Пример файла app.py:

```

from aiohttp import web
import aiohttp_jinja2
import jinja2

# Инициализируем приложение aiohttp
app = web.Application()

# Настраиваем Jinja2 для использования шаблонов из
папки Templates
aiohttp_jinja2.setup(app,
loader=jinja2.FileSystemLoader('Templates'))

# URL адрес сервиса "user"
USER_SERVICE_URL = 'http://user_service:5002'

# Обработчик главной страницы
@aiohttp_jinja2.template('index.html')
async def handle_index(request):
    return {}

# Обработчик страницы задач
@aiohttp_jinja2.template('tasks.html')
async def handle_tasks(request):
    if request.method == 'GET':
        # Обработка GET запроса для страницы задач
        return {}

# Обработчик страницы регистрации
@aiohttp_jinja2.template('register.html')

```

```

async def handle_register(request):
    if request.method == 'GET':
        # Обработка GET запроса для страницы
        # регистрации
        return {}

# Обработчик создания задачи
async def create_task(request):
    user_id = request.match_info.get('user_id')
    task_data = await request.json()

    async with aiohttp.ClientSession() as session:
        url = f'http://user_service:5002/user/{user_id}'
        async with session.get(url) as resp:
            user_info = await resp.json()
            print("User info:", user_info)
            # Здесь можно использовать информацию о
            # пользователе для создания задачи
            return web.json_response({'message': 'Task
created successfully'})

# Обработчик пользователя
async def handle_user(request):
    user_id = request.match_info.get('user_id')
    user_info = {'user_id': user_id, 'username':
'test_user'}
    return web.json_response(user_info)

# Регистрируем обработчики для соответствующих страниц
app.router.add_get('/', handle_index)
app.router.add_get('/tasks', handle_tasks)
app.router.add_get('/register', handle_register)
app.router.add_post('/task/{user_id}', create_task)
app.router.add_get('/user/{user_id}', handle_user)

# Запускаем веб-сервер
if __name__ == "__main__":
    web.run_app(app, port=5000, host='0.0.0.0')

```

В данном приложении для реализации асинхронной коммуникации вместо синхронных HTTP запросов к другим сервисам, были реализованы

асинхронные API, где запросы отправляются асинхронно, а ответы получаются позже через callback функции или события.

В случае возникновения трудностей, убедитесь, что ваши контейнеры находятся в одной Docker-сети. Также при необходимости в файлах requirements необходимо указать нужные инструменты реализации.

Вопросы к практической работе

1. Что такое асинхронная коммуникация между микросервисами и в чем ее преимущества?
2. Какие виды асинхронной коммуникации используются в микросервисной архитектуре?
3. Какие проблемы решает асинхронная коммуникация по сравнению с синхронной?
4. Какие технологии и протоколы используются для реализации асинхронной коммуникации между микросервисами?
5. Какие сценарии использования наиболее подходят для асинхронной коммуникации?
6. Как обрабатываются ошибки и сбои в асинхронной коммуникации между микросервисами?
7. Какие особенности асинхронной коммуникации нужно учитывать при проектировании микросервисов?
8. Какие инструменты и библиотеки можно использовать для упрощения реализации асинхронной коммуникации?
9. Какие компоненты инфраструктуры микросервисов должны быть настроены для поддержки асинхронной коммуникации?
10. Какие могут быть недостатки или ограничения при использовании асинхронной коммуникации между микросервисами?

Критерии оценки

- Реализована асинхронная коммуникация между микросервисами
- Проверена работа приложения
- Составлен отчет о проделанной работе со скриншотами её выполнения
- Даны ответы на вопросы к практической работе